

SpotifUM – Sistema de Gestão de Música

Programação Orientada aos Objetos

2024/2025

Diogo Pais A107368

João Gomes A107377

Pedro Pereira A107332

Índice

1. Introdução	3
2. Arquitetura da aplicação	4
2.1. Diagrama de Classes	4
2.2 Justificação do Diagrama de Classes	4
3. Funcionalidades Desenvolvidas	6
4. Persistência de Dados	8
5. Exemplos de Execução	9
6. Conclusão	12

1. Introdução

O projeto SpotifUM é uma aplicação desenvolvida no âmbito da unidade curricular de Programação Orientada aos Objetos. Esta aplicação permite aos utilizadores ouvir músicas, criar playlists, acumular pontos de acordo com o seu plano de subscrição e interagir com um sistema de reprodução simplificado.

A aplicação segue os princípios da programação orientada aos objetos, com uma estrutura modular e extensível. Está organizada em torno das principais entidades do domínio musical: músicas, playlists, utilizadores, planos de subscrição e álbuns. Cada uma destas entidades é representada por uma ou mais classes, de forma a facilitar a escalabilidade e manutenção do sistema.

O SpotifUM suporta múltiplos tipos de utilizadores (free, premiumBase e premiumTop), distintos tipos de músicas (incluindo explícitas e multimédia) e várias categorias de playlists (aleatórias, favoritas, premium). Para além disso, a aplicação inclui funcionalidades avançadas como estatísticas de reprodução, pontuação dos utilizadores, reprodução interativa por menus, e persistência de dados.

A reprodução de músicas é simulada por meio da apresentação do texto da letra da música na consola, e o sistema mantém um registo do número de vezes que cada música foi ouvida, contribuindo para funcionalidades como playlists personalizadas baseadas em gostos.

Este relatório documenta as decisões tomadas no desenho da arquitetura de classes, a implementação das funcionalidades principais e as abordagens seguidas para garantir a modularidade, reutilização e robustez do código.

Cada utilizador é representado pela classe User, que contém atributos como nome, email, morada e um conjunto de relações relevantes: um PlanoSubscricao, uma

Biblioteca e as Playlist que cria. A relação entre User e PlanoSubscricao é de agregação, o que permite a partilha e substituição de planos sem comprometer a identidade do utilizador. A interface PlanoSubscricao é implementada por três subclasses concretas — PlanoFree, PlanoPremiumBase e PlanoPremiumTop — que definem comportamentos distintos ao nível de permissões e sistema de pontuação. Esta escolha favorece o polimorfismo e torna o sistema aberto à introdução de novos planos no futuro, respeitando o princípio Open/Closed.

Cada utilizador possui ainda uma Biblioteca, que contém múltiplos Album e Playlist. O Album, por sua vez, é composto por uma lista de Musica, permitindo a organização de faixas por parte dos utilizadores. A classe Musica representa uma faixa sonora individual, com dados como nome, artista, letra e editora, e está associada a uma instância de TipoMusica, uma classe abstrata que descreve o tipo da música. Através desta associação, é possível representar músicas explícitas (MusicaExplicita) e multimédia (MusicaMultimedia) sem alterar a estrutura de Musica, promovendo assim o uso de composição em vez de herança direta e permitindo comportamentos distintos através de polimorfismo.

A componente de playlists foi modelada com a classe abstrata Playlist, que define a estrutura base para listas de reprodução. A partir dela derivam três tipos específicos: PlaylistAleatoria, PlaylistPremium e PlaylistFavoritos. As PlaylistAleatoria são utilizadas por utilizadores com plano gratuito e reproduzem músicas numa ordem aleatória. Já as PlaylistPremium, associadas a utilizadores premium, permitem funcionalidades adicionais como avançar e retroceder faixas. As PlaylistFavoritos são geradas automaticamente com base nas músicas mais reproduzidas pelo utilizador. Todas estas playlists estão associadas a uma instância de User através de relações de associação, refletindo a ideia de propriedade ou autoria sobre o conteúdo.

A análise de dados está centralizada na classe Estatisticas, que depende da informação de User e de Musica para gerar relatórios como a música mais reproduzida ou o utilizador com mais pontos. Esta separação do módulo estatístico da lógica principal contribui para a aplicação do princípio da responsabilidade única, tornando o sistema mais fácil de manter e evoluir.

O diagrama utiliza diferentes tipos de relações para expressar a estrutura do sistema de forma precisa: herança (entre classes como PlanoFree e PlanoSubscricao), agregação (entre User e Biblioteca, ou User e Playlist), e associações (entre Playlist e Musica, Album e Musica, entre outras).

Com esta modelação, o sistema SpotifUM consegue refletir com clareza as entidades e interações necessárias para gerir músicas, utilizadores e planos de subscrição, oferecendo ao mesmo tempo uma base sólida para futuras expansões e melhorias.

3. Funcionalidades Desenvolvidas

O sistema SpotifUM foi concebido para oferecer uma experiência completa de gestão e reprodução de conteúdos musicais, adaptada a diferentes tipos de utilizadores e planos de subscrição. A implementação das funcionalidades teve por base os requisitos especificados no enunciado, organizando o sistema em torno de operações intuitivas e acessíveis através de menus. Esta secção descreve as principais funcionalidades implementadas.

Uma das funcionalidades centrais é a criação e gestão de utilizadores. Os utilizadores são criados com dados pessoais e associados de imediato a um plano de subscrição: PlanoFree, PlanoPremiumBase ou PlanoPremiumTop. Cada plano oferece diferentes capacidades de interação com o sistema, nomeadamente ao nível da reprodução de playlists e da acumulação de pontos. Esta diferenciação foi feita com recurso ao polimorfismo, permitindo tratar utilizadores de diferentes planos de forma uniforme, mas respeitando as particularidades de cada plano.

O sistema permite ainda adicionar e gerir músicas, com suporte para diferentes tipos como MusicaExplicita e MusicaMultimedia, através da abstração TipoMusica. Cada música inclui atributos como nome, artista, letra, pauta e contador de reproduções. Estas músicas podem ser agrupadas em álbuns, os quais são adicionados à biblioteca pessoal do utilizador. A estrutura da biblioteca permite o acesso organizado aos conteúdos, sendo possível consultar, reproduzir ou adicionar músicas diretamente a playlists.

A funcionalidade de criação e gestão de playlists está disponível para todos os utilizadores. As playlists podem ser do tipo PlaylistAleatoria, utilizadas por utilizadores Free com reprodução em ordem aleatória, ou PlaylistPremium, disponíveis para utilizadores premium e que oferecem controlo adicional sobre a ordem de reprodução, permitindo avançar e retroceder faixas. As PlaylistFavoritos são geradas automaticamente com base nas estatísticas de reprodução, reunindo as músicas mais ouvidas por cada utilizador. Cada playlist pode ser pública ou privada, influenciando a sua visibilidade no sistema.

A reprodução de músicas e playlists foi implementada através da simulação textual, em que a letra da música é apresentada progressivamente no terminal, linha a linha. Esta simulação substitui a reprodução real de áudio e permite distinguir comportamentos entre utilizadores com diferentes permissões, como o controlo de avanço ou acesso a playlists privadas.

O sistema integra ainda um módulo de estatísticas, gerido pela classe Estatisticas, que permite determinar a música mais reproduzida, o utilizador com maior pontuação, e outros indicadores relevantes. Estas estatísticas não só contribuem para

a personalização da experiência (por exemplo, na geração de playlists favoritas), como também podem ser utilizadas para futuros desenvolvimentos como recomendações automáticas ou rankings globais.

Para além disso, foram implementadas funcionalidades de pesquisa por nome de música, artista ou género, bem como a possibilidade de remover conteúdos, editar letras, associar pautas e visualizar álbuns completos. O sistema regista automaticamente a quantidade de vezes que cada música foi ouvida, o que contribui para o sistema de pontuação dos utilizadores e para as estatísticas globais.

Finalmente, a aplicação permite a salvaguarda e carregamento do estado do sistema através de serialização, garantindo que todos os dados persistem entre sessões. Esta funcionalidade é acessível a partir do menu principal e oferece uma forma robusta de manter a integridade do sistema ao longo do tempo.

Todas estas funcionalidades são acessíveis por um menu interativo baseado em consola, desenvolvido nas classes `Menu` e `NewMenu`, e organizadas de forma a separar a lógica da interface da lógica de negócio. Esta abordagem garante modularidade e clareza no código, além de facilitar a manutenção futura.

4. Persistência de Dados

Para garantir que o estado do sistema é preservado entre diferentes sessões de utilização, a aplicação SpotifUM implementa um mecanismo de persistência de dados através de serialização. Esta funcionalidade permite guardar toda a informação relevante — como utilizadores, músicas, álbuns, playlists e estatísticas — num ficheiro externo, que pode ser posteriormente lido para restaurar o sistema exatamente como estava antes.

A serialização foi implementada recorrendo à API de entrada/saída de objetos (ObjectOutputStream e ObjectInputStream) da linguagem Java. A classe principal SpotifyUM foi declarada como Serializable, bem como todas as suas componentes internas que contêm estado relevante, nomeadamente as classes User, Musica, Album, Playlist, Biblioteca e Estatisticas. Isto assegura que toda a estrutura interna do sistema pode ser escrita e lida diretamente de um ficheiro binário.

No final de cada execução, o sistema oferece ao utilizador a opção de guardar o estado atual da aplicação. Quando selecionada, esta operação invoca um método responsável por escrever o objeto principal SpotifyUM para um ficheiro — por exemplo, estado_spotifum.dat. Este ficheiro é criado ou substituído automaticamente e contém toda a estrutura de dados necessária para restaurar o estado completo do sistema.

De forma análoga, no arranque do programa, o sistema verifica a existência deste ficheiro e, se presente, carrega automaticamente o estado guardado. Esta recuperação permite retomar o funcionamento do sistema a partir do ponto exato em que foi encerrado, incluindo utilizadores já registados, playlists criadas, músicas ouvidas e estatísticas acumuladas.

Este mecanismo foi encapsulado em métodos específicos dentro da classe SpotifyUM, nomeadamente guardarEstado() e carregarEstado(), promovendo a separação de responsabilidades e mantendo o código de persistência isolado da lógica principal da aplicação. A manipulação de exceções foi também considerada, garantindo que erros de leitura ou escrita são tratados de forma apropriada, evitando que o sistema entre em estados inconsistentes.

A utilização da serialização binária revelou-se uma solução simples e eficaz para o contexto do projeto, permitindo a persistência transparente do estado do sistema com esforço mínimo de implementação. Caso o sistema venha a evoluir para uma aplicação de maior escala, este mecanismo poderá ser facilmente substituído por soluções mais robustas, como bases de dados relacionais ou formatos de armazenamento estruturado (ex: JSON, XML), sem necessidade de alterações significativas na lógica de negócio.

5. Exemplos de Execução

A aplicação SpotifUM foi desenvolvida com uma interface textual simples e intuitiva, permitindo ao utilizador navegar por menus e realizar todas as operações de forma interativa. Nesta secção apresentam-se alguns exemplos representativos da execução da aplicação, que demonstram o fluxo típico de utilização, desde a criação de entidades até à reprodução de músicas e consulta de estatísticas.

Início da aplicação:

```
--- Bem-vindo ao SpotifUM ---  
1 - Entrar como Admin  
2 - Entrar como Utilizador  
0 - Sair  
Opção:
```

Figura 2: Ecrã de abertura da aplicação

Ao abrirmos a nossa aplicação temos a opção para entrar como Admin ou como Utilizador. Vamos verificar o que acontece em cada um. Começando pelo menu Admin.

```
*** NewMenu ***  
1 - Gestão de Utilizadores  
2 - Gestão de Álbuns  
3 - Gestão de Playlists  
4 - Estatísticas  
5 - Guardar Estado  
6 - Carregar Estado  
0 - Sair  
Opção: |
```

Figura 3: Menu Admin

No menu Admin temos as 6 opções que verificamos na figura 3. A nossa primeira opção serve para gerirmos os utilizadores já guardados no nosso sistema, podendo mudar os seus planos ou apenas listá-los e por fim podemos adicionar utilizadores.

A nossa segunda opção serve para gerir os álbuns onde o administrador pode escolher adicionar um album ou apenas listá-los também. A opção de gestão de playlists infelizmente ainda não foi implementada.

Ao entrarmos na opção Estatísticas vamos para outro submenu onde podemos ver as estatísticas da nossa aplicação, conseguindo saber qual a música mais reproduzida, o utilizador com mais pontos e entre outras coisas.

```
*** NewMenu ***
1 - Música mais reproduzida
2 - Utilizador com mais pontos
3 - Artista mais reproduzido
4 - Utilizador com mais reproduções
5 - Género mais reproduzido
6 - Utilizador com mais playlists
7 - Número total de playlists públicas
0 - Sair
Opção:
```

Figura 4: Submenu Estatísticas

Voltando ao nosso menu Admin podemos verificar os últimos dois pontos que são utilizados para guardar o estado da aplicação naquele momento e a opção para carregar o estado anteriormente guardado.

```
*** NewMenu ***
1 - Criar Playlist com base nos Favoritos
2 - Ouvir playlists públicas
3 - Biblioteca
4 - Criar playlist
5 - Tornar playlist pública
0 - Sair
Opção: |
```

Figura 5: Menu Utilizador PremiumTop

Para entrarmos no menu utilizador precisamos de fazer um login com o email e password, dependendo do plano do utilizador as opções ao entrarmos são diferentes. Para um utilizador PremiumTop temos 5 opções, onde podemos criar

playlists com base nos favoritos, ouvir playlists públicas, a sua biblioteca de playlists e criar playlist.

```
*** NewMenu ***  
1 - Ouvir playlists públicas aleatórias  
0 - Sair  
Opção:
```

Figura 6: Menu Utilizador Free

O utilizador com plano Free apenas consegue ouvir playlists públicas aleatórias.

```
*** NewMenu ***  
1 - Ouvir playlists públicas  
2 - Biblioteca  
3 - Criar playlist  
4 - Tornar playlist pública  
0 - Sair  
Opção:
```

Figura 7: Menu Utilizador PremiumBase

O utilizador com plano PremiumBase tem as mesmas opções que um utilizador com plano PremiumTop exceto criar playlist com base nos favoritos

6. Conclusão

O desenvolvimento do projeto SpotifUM permitiu consolidar os principais conceitos da programação orientada aos objetos, aplicando-os de forma prática na construção de um sistema completo de gestão musical. Através da definição cuidada da arquitetura de classes e da utilização de princípios como herança, polimorfismo e encapsulamento, foi possível criar uma aplicação modular, reutilizável e preparada para evolução futura.

A implementação de funcionalidades como a criação de utilizadores, reprodução de músicas, geração de playlists personalizadas e análise estatística demonstrou a robustez da estrutura construída, bem como a sua capacidade de adaptação a diferentes cenários de utilização. A introdução da persistência de dados via serialização contribuiu para a continuidade do sistema entre sessões, reforçando a sua utilidade prática.

Em suma, o SpotifUM cumpre os objetivos propostos, oferecendo uma base sólida para um sistema de gestão musical orientado a objetos. Com base nesta estrutura, seria possível no futuro introduzir melhorias como uma interface gráfica, integração com ficheiros de áudio reais, ou ligação a uma base de dados externa, tornando a aplicação ainda mais completa e realista.